

Static Trees, RMQ y Strings para el Final de EDA

Guia de teoria y clasificacion de problemas basada en los slides locales

Como usar esta guia

Esta guia esta hecha para estudiar rapido, no para leer pasivamente los slides. El examen es a papel y los problemas son estilo Codeforces/Yosupo, pero la respuesta esperada suele ser teorica: idea, estructura, pseudocodigo, correctitud y complejidad. Por eso, para cada tema se enfatiza:

- que problema reconoce la estructura;
- que estructura usar;
- que invariante/proof escribir;
- que complejidad cerrar.

La carpeta fuente es `statics and strings/`. Los PDFs cubiertos son:

- `eda_slides_08_rmq.pdf`: static trees, RMQ, LCA, level ancestor.
- `Estructuras_de_Datos_Avanzadas___Strings_I.pdf`: RMQ/LCA y pattern matching.
- `Estructuras_de_Datos_Avanzadas___Strings_II.pdf`: suffix array, DC3, SA-IS.
- `CS3014___Strings.pdf`: nota larga de suffix array, Manber–Myers, DC3, Ko-Aluru/SAIS.

Contents

1	Mapa mental para clasificar problemas	4
1.1	Primera pregunta: es array, arbol o string?	4
1.2	Segunda pregunta: hay updates?	4
1.3	Tercera pregunta: la consulta es por rango, camino o prefijo?	4
1.4	Checklist de respuesta de examen	4
2	Diccionario fundamental desde cero	5
2.1	Preprocesamiento	5
2.2	Query o consulta	5
2.3	Operacion idempotente	5
2.4	Sparse table	5
2.5	Tabla de logs	5
2.6	Arbol enraizado, ancestro y profundidad	6
2.7	LCA	6
2.8	Binary lifting	6
2.9	Euler tour	6
2.10	Rank en strings	6
2.11	Suffix array	7
2.12	LCP	7
2.13	Sentinela	7
2.14	Separadores entre strings	7
2.15	Como decidir rapido que definicion necesitas	7

3	RMQ en arrays estaticos	8
3.1	Definicion	8
3.2	Sparse table: la version que debes dominar	8
3.3	Cuando sparse table NO sirve	8
3.4	Como escribir sparse table en una solucion teorica	9
3.5	RMQ lineal: macro/micro bloques	9
3.6	Hint de clasificacion	9
4	Cartesian trees: puente entre RMQ y LCA	10
4.1	Definicion	10
4.2	Identidad clave	10
4.3	Correctitud	10
4.4	Construccion lineal	10
4.5	Hint de clasificacion	10
5	LCA y su reduccion a RMQ	11
5.1	Definicion	11
5.2	Por que LCA resuelve caminos	11
5.3	Solucion simple: binary lifting	11
5.4	Correctitud de binary lifting	11
5.5	Binary lifting para subir k niveles	12
5.6	Reduccion a RMQ con Euler tour	12
5.7	Propiedad ± 1	12
5.8	Correctitud	12
5.9	Hint de clasificacion	12
5.10	Como decidir entre las dos soluciones de LCA	13
6	Level Ancestor	14
6.1	Definicion	14
6.2	Soluciones vistas en slides	14
6.3	Que debes saber para examen	14
6.4	Hint de clasificacion	14
7	Strings: notacion basica	15
7.1	Conceptos	15
7.2	Subcadena, subsecuencia, prefijo y sufijo	15
7.3	Por que importa el sentinel	15
7.4	Orden lexicografico	15
7.5	Idea clave para problemas	15
7.6	Hint de clasificacion	15
8	Suffix array	16
8.1	Definicion	16
8.2	Por que sirve	16
8.3	Rank inverso	16
8.4	Aplicaciones de examen	17
8.5	Hint de clasificacion	17
9	Manber–Myers: construccion $O(n \log n)$	18
9.1	Idea central	18
9.2	Que es una clase o rank temporal	18
9.3	Paso inductivo	18
9.4	Pseudocodigo teorico	18
9.5	Optimizacion del slide	18
9.6	Correctitud	18
9.7	Complejidad	18

10 LCP array y RMQ sobre LCP	19
10.1 Definicion	19
10.2 Por que solo vecinos	19
10.3 LCP arbitrario entre dos sufijos	19
10.4 Correctitud	19
10.5 Usos	19
10.6 Plantilla de examen para SA + LCP	19
10.7 Hint de clasificacion	20
11 Pattern matching	21
11.1 Dos escenarios de los slides	21
11.2 Texto fijo + patrones	21
11.3 Muchos patrones	21
11.4 Correctitud del intervalo	21
11.5 Complejidad	21
12 Contar subcadenas distintas	22
12.1 Formula	22
12.2 Por que funciona	22
12.3 Hint de clasificacion	22
12.4 Complejidad	22
13 Longest common substring	23
13.1 Problema	23
13.2 Algoritmo	23
13.3 Correctitud	23
13.4 Complejidad	23
14 Ordenar y comparar subcadenas	24
14.1 Problema	24
14.2 Estructura	24
14.3 Comparacion	24
14.4 Complejidad	24
15 DC3	25
15.1 Idea	25
15.2 Por que sirve	25
15.3 Pasos	25
15.4 Complejidad	25
15.5 Hint de examen	25
16 SA-IS y clasificacion S/L	26
16.1 Idea	26
16.2 Teorema importante	26
16.3 Por que es lineal	26
16.4 Hint de examen	26
17 Bordes, KMP y periodicidad	27
17.1 Bordes	27
17.2 Periodicidad	27
17.3 Hint de clasificacion	27
18 Problemas tipo examen y estructura a usar	28
19 Mapa de slides cubiertos	29
20 Apendice: mapa pagina por pagina de los slides	30

1 Mapa mental para clasificar problemas

1.1 Primera pregunta: es array, arbol o string?

Antes de pensar en algoritmos, clasifica el objeto principal:

Objeto	Frases tipicas	Estructuras candidatas
Array estatico	rangos, minimo, maximo, no hay updates	Sparse table, RMQ lineal macro/micro
Arbol estatico	LCA, ancestro, camino, subarbol	Euler tour, binary lifting, RMQ, HLD conceptual
String texto fijo	buscar patrones, contar ocurrencias, subcadenas	Suffix array, LCP, RMQ sobre LCP
Muchos patrones	varios patrones contra un texto	Aho-Corasick o suffix array por intervalos
Substrings ordenadas	comparar/ordenar muchas subcadenas	SA + rank + LCP + RMQ

1.2 Segunda pregunta: hay updates?

Los temas de esta carpeta son principalmente estaticos. Si el input no cambia, preprocesa fuerte. Si hay updates, cuidado:

- Array con updates: ya no basta sparse table; piensa segment tree.
- Arbol fijo con path queries: HLD + segment tree puede aparecer.
- String fijo con muchas queries: suffix array sirve.
- String dinamico: no es el foco de estos slides.

1.3 Tercera pregunta: la consulta es por rango, camino o prefijo?

- Rango en array: sparse table/RMQ.
- Camino en arbol: LCA, HLD o transformar a RMQ.
- Subarbol: Euler tour convierte a intervalo.
- Patron en string: intervalo contiguo en suffix array.
- Comparar dos substrings: LCP entre sufijos + RMQ sobre LCP.

1.4 Checklist de respuesta de examen

Para casi cualquier problema de estos temas escribe:

1. **Modelo del problema.** Que se preprocesa y que se consulta.
2. **Estructura.** Sparse table, Cartesian tree, Euler tour, SA, LCP, etc.
3. **Invariante.** Que significa cada arreglo/tabla.
4. **Consulta.** Como se responde paso a paso.
5. **Correctitud.** Por que cubre todos los casos.
6. **Complejidad.** Preprocesamiento, query, espacio.

2 Diccionario fundamental desde cero

Esta seccion existe porque en examen no basta decir “uso sparse table” o “uso binary lifting”. Debes poder explicar que es la estructura, que guarda, por que responde la consulta y por que su complejidad es la que dices.

2.1 Preprocesamiento

Preprocesar significa gastar tiempo antes de recibir o responder consultas para construir informacion auxiliar. La idea central de las estructuras estaticas es: si el arreglo, arbol o string no cambia, conviene calcular tablas una sola vez para responder muchas preguntas rapidamente.

Ejemplo: si te dan un arreglo fijo y 10^5 consultas de minimo en rango, no conviene recorrer el rango completo en cada consulta. Conviene construir una tabla de respuestas parciales.

2.2 Query o consulta

Una query es una pregunta que se hace sobre la estructura ya construida. En estos temas las queries tipicas son:

- “cual es el minimo de $A[l..r]$?”
- “cual es el LCA de u y v ?”
- “cuantas veces aparece el patron P en el texto S ?”
- “cual es el LCP de dos sufijos?”

2.3 Operacion idempotente

Una operacion es idempotente si repetir un elemento no cambia la respuesta:

$$\min(x, x) = x, \quad \max(x, x) = x, \quad \gcd(x, x) = x.$$

Esto es fundamental para sparse table. En una consulta de minimo podemos cubrir un intervalo con dos bloques que se solapan, porque si un elemento queda contado dos veces no altera el minimo. Para suma esto no funciona: $x + x \neq x$.

2.4 Sparse table

Una **sparse table** es una tabla de preprocesamiento para arreglos estaticos. Guarda respuestas de rangos cuya longitud es potencia de dos:

$$1, 2, 4, 8, 16, \dots$$

No guarda todos los rangos posibles, porque eso seria $O(n^2)$. Guarda solo rangos “canonicos”:

$$st[k][i] = \text{respuesta del rango que empieza en } i \text{ y mide } 2^k.$$

Para minimos:

$$st[k][i] = \min(A[i], A[i+1], \dots, A[i+2^k-1]).$$

La razon por la que sirve es que cualquier rango $[l, r]$ contiene una potencia de dos grande. Si $2^k \leq r - l + 1$, entonces dos bloques de longitud 2^k cubren todo el intervalo: uno pegado a la izquierda y otro pegado a la derecha.

2.5 Tabla de logs

La sparse table suele necesitar:

$$\log[len] = \lfloor \log_2(len) \rfloor.$$

Esto se puede precomputar con:

$$\log[1] = 0, \quad \log[i] = \log[\lfloor i/2 \rfloor] + 1.$$

En papel no necesitas codigo, pero si debes saber que obtener k para una consulta cuesta $O(1)$ si ya tienes esta tabla.

2.6 Arbol enraizado, ancestro y profundidad

Un arbol enraizado es un arbol donde escoges un nodo como raiz. Desde ahi se define:

- **padre**: el nodo anterior en el camino hacia la raiz;
- **ancestro**: un nodo que aparece al subir desde v hacia la raiz;
- **profundidad**: numero de aristas desde la raiz hasta el nodo;
- **subarbol**: un nodo y todos sus descendientes.

2.7 LCA

El **Lowest Common Ancestor** de u y v es el ancestro comun de ambos que esta mas abajo, es decir, el de mayor profundidad. Si uno de los nodos es ancestro del otro, entonces el LCA es ese mismo nodo.

Por que aparece tanto: muchas preguntas sobre caminos en arbol se parten en dos tramos usando el LCA:

$$u \rightarrow \text{LCA}(u, v) \quad \text{y} \quad \text{LCA}(u, v) \rightarrow v.$$

2.8 Binary lifting

Binary lifting significa subir en potencias de dos. En lugar de guardar solo el padre inmediato de cada nodo, guardas:

$$\begin{aligned} \text{up}[v][0] &= \text{padre de } v, \\ \text{up}[v][1] &= \text{ancestro 2 niveles arriba}, \\ \text{up}[v][2] &= \text{ancestro 4 niveles arriba}, \end{aligned}$$

y en general:

$$\text{up}[v][j] = \text{ancestro } 2^j \text{ niveles arriba de } v.$$

La formula de construccion es:

$$\text{up}[v][j] = \text{up}[\text{up}[v][j-1]][j-1].$$

Se lee asi: para subir 2^j , sube primero 2^{j-1} y desde ahi sube otros 2^{j-1} .

La idea funciona porque cualquier numero k se puede escribir como suma de potencias de dos. Por ejemplo, para subir 13 niveles:

$$13 = 8 + 4 + 1.$$

Entonces saltas 8, luego 4, luego 1.

2.9 Euler tour

Un **Euler tour** de un arbol enraizado es la secuencia de nodos visitados por un DFS cuando entras y cuando vuelves de hijos. Para LCA no se guarda solo la primera visita: se guarda la caminata completa. Tambien se guarda la profundidad de cada visita.

Ejemplo conceptual: si al recorrer el arbol pasas por nodos

$$E = [0, 1, 3, 1, 4, 1, 0, 2, 0],$$

entonces guardas tambien:

$$D = [0, 1, 2, 1, 2, 1, 0, 1, 0].$$

El LCA de dos nodos se convierte en buscar la minima profundidad entre sus primeras apariciones.

2.10 Rank en strings

En suffix array, **rank[i]** significa la posicion lexicografica del sufijo que empieza en i . Si $\text{rank}[i] < \text{rank}[j]$, entonces:

$$S[i..] < S[j..].$$

El rank es el inverso del suffix array:

$$\text{SA}[\text{rank}[i]] = i.$$

2.11 Suffix array

El **suffix array** es un arreglo que lista todos los sufijos de un string en orden lexicografico. Si:

$$S = \text{banana}\$,$$

sus sufijos son:

$$\text{banana}\$, \text{anana}\$, \text{nana}\$, \text{ana}\$, \text{na}\$, \text{a}\$, \$.$$

Ordenados lexicograficamente quedan:

$$\$, \text{a}\$, \text{ana}\$, \text{anana}\$, \text{banana}\$, \text{na}\$, \text{nana}\$.$$

El suffix array guarda sus posiciones iniciales:

$$\text{SA} = [6, 5, 3, 1, 0, 4, 2].$$

2.12 LCP

LCP significa longest common prefix: prefijo comun mas largo. Por ejemplo:

$$\text{lcp}(\text{anana}\$, \text{ana}\$) = 3$$

porque ambas empiezan con **ana**.

El arreglo LCP del suffix array guarda el LCP entre sufijos vecinos en orden lexicografico. Esto es poderoso porque muchos problemas de substrings repetidas se vuelven preguntas sobre vecinos o minimos en rangos del LCP.

2.13 Sentinela

El sentinela **\$** es un caracter artificial menor que todos los demas y se pone al final del string. Sirve para que todos los sufijos sean comparables sin casos ambiguos. Tambien asegura que ningun sufijo sea igual a otro.

2.14 Separadores entre strings

Cuando concatenas dos strings, por ejemplo para longest common substring, usas separadores que no aparecen en ningun string:

$$T = A\#B\$.$$

Esto evita que una subcadena cruce artificialmente del final de *A* al inicio de *B*.

2.15 Como decidir rapido que definicion necesitas

Palabra del enunciado	Definicion que debes activar
rango estatico	sparse table, RMQ, operacion idempotente
ancestro, camino en arbol	LCA, profundidad, binary lifting
subir <i>k</i> niveles	binary lifting, level ancestor
subarbol	Euler tour con tiempos de entrada/salida
patron en texto fijo	suffix array, intervalo lexicografico
substring repetida	LCP, vecinos en suffix array
comparar substrings	rank, LCP arbitrario, RMQ sobre LCP
dos strings	separadores, suffix array conjunto, origen de sufijo

3 RMQ en arrays estaticos

3.1 Definicion

El problema **Range Minimum Query** recibe un arreglo fijo $A[0..n-1]$. Una consulta $\text{RMQ}(l, r)$ pide el indice o valor minimo en:

$$A[l], A[l+1], \dots, A[r].$$

El slide de RMQ enfatiza tres metas:

- construccion $O(n \log n)$ y query $O(1)$;
- construccion $O(n)$ y query $O(\log n)$;
- construccion $O(n)$ y query $O(1)$.

3.2 Sparse table: la version que debes dominar

La sparse table es una matriz donde las filas representan tamanos de bloque y las columnas representan posiciones iniciales. La fila 0 guarda bloques de tamano 1, la fila 1 bloques de tamano 2, la fila 2 bloques de tamano 4, y asi sucesivamente.

La sparse table guarda respuestas para bloques de longitud potencia de dos:

$$st[k][i] = \min(A[i], \dots, A[i + 2^k - 1]).$$

La transicion:

$$st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}]).$$

Consulta

Para responder $[l, r]$:

$$k = \lfloor \log_2(r - l + 1) \rfloor.$$

Usas dos bloques de longitud 2^k :

$$[l, l + 2^k - 1] \quad \text{y} \quad [r - 2^k + 1, r].$$

La respuesta:

$$\min(st[k][l], st[k][r - 2^k + 1]).$$

Textualmente: tomo el bloque mas grande de potencia de dos que cabe dentro de la consulta. Pongo una copia de ese bloque empezando en l y otra terminando en r . Entre ambos cubren todo $[l, r]$. Como estoy usando minimo, no importa si se superponen.

Por que funciona

Los dos bloques cubren todo el intervalo. Pueden solaparse, pero eso no importa porque min es idempotente:

$$\min(x, x) = x.$$

Este argumento es el que debes escribir en examen.

Complejidad

$$\text{Preproceso } O(n \log n), \quad \text{query } O(1), \quad \text{espacio } O(n \log n).$$

3.3 Cuando sparse table NO sirve

Sparse table no soporta updates faciles. Si el problema dice:

- cambia $A[i]$;
- suma en rango;
- asigna valores;

entonces piensa en segment tree, no en sparse table. Sparse table es para datos estaticos y operaciones idempotentes como min/max/gcd.

3.4 Como escribir sparse table en una solucion teorica

Una respuesta fuerte en papel puede decir:

1. Preproceso $st[k][i]$, el minimo del subarreglo de longitud 2^k que empieza en i .
2. Caso base: $st[0][i] = A[i]$.
3. Transicion: parto el bloque de longitud 2^k en dos bloques de longitud 2^{k-1} .
4. Para consultar $[l, r]$, tomo $k = \lfloor \log_2(r - l + 1) \rfloor$ y combino los dos bloques extremos.
5. Correctitud: los bloques cubren el intervalo y el solapamiento no afecta por idempotencia del minimo.
6. Complejidad: $O(n \log n)$ construccion, $O(1)$ query, $O(n \log n)$ memoria.

3.5 RMQ lineal: macro/micro bloques

Los slides de static trees mencionan la meta optima $O(n)$ espacio y $O(1)$ query. La idea:

1. Partir el arreglo en bloques de tamano $b = \frac{1}{2} \log n$.
2. Macroestructura: sparse table sobre minimos de bloques.
3. Microestructura: precomputar respuestas dentro de cada forma de bloque.

En ± 1 RMQ, cada bloque tiene pocas formas posibles: $2^{b-1} = O(\sqrt{n})$. Por eso se pueden precomputar tablas internas en $o(n)$ espacio.

3.6 Hint de clasificacion

Si ves:

- “static RMQ”;
- “range minimum, no updates”;
- “preprocess once, many queries”;

usa sparse table salvo que pidan explicitamente espacio lineal. Si piden optimalidad teorica, menciona macro/micro.

4 Cartesian trees: puente entre RMQ y LCA

4.1 Definicion

El Cartesian tree de un arreglo A es un arbol binario que cumple:

- propiedad de heap: la raiz es el minimo del arreglo;
- inorder traversal preserva el orden de indices;
- el subarbol izquierdo se construye con elementos a la izquierda del minimo;
- el subarbol derecho con elementos a la derecha.

4.2 Identidad clave

Para indices $i \leq j$:

$$\text{RMQ}_A(i, j) = \text{LCA}_T(i, j)$$

donde T es el Cartesian tree.

4.3 Correctitud

El minimo de $A[i..j]$ es el elemento de menor valor entre esos indices. En el Cartesian tree, por la propiedad heap, ese elemento queda por encima de todos los elementos del intervalo. Por la propiedad inorder, separa exactamente el intervalo en izquierda y derecha. Por eso es el ancestro comun mas bajo de los nodos i y j .

4.4 Construccion lineal

El slide menciona el algoritmo con **right spine**:

1. Escanear el arreglo de izquierda a derecha.
2. Mantener la espina derecha del Cartesian tree parcial.
3. Para insertar $A[i]$, subir por la espina hasta encontrar un nodo con valor menor.
4. El antiguo subarbol desplazado pasa a ser hijo izquierdo del nuevo nodo.

Cada nodo entra y sale de la espina a lo mucho una vez, por eso el costo es amortizado $O(n)$.

4.5 Hint de clasificacion

Si un problema te pide demostrar equivalencia entre RMQ y LCA, o construir una estructura optima para RMQ, menciona Cartesian tree. No es la primera herramienta para implementar rapido, pero si es una herramienta teorica fuerte.

5 LCA y su reduccion a RMQ

5.1 Definicion

El **Lowest Common Ancestor** de dos nodos u, v en un arbol enraizado es el ancestro comun mas profundo de ambos.

Ejemplo textual: si el camino desde la raiz hasta u es

$$raiz \rightarrow a \rightarrow b \rightarrow u$$

y el camino desde la raiz hasta v es

$$raiz \rightarrow a \rightarrow c \rightarrow v,$$

entonces los ancestros comunes son $raiz$ y a . El mas profundo es a , por lo tanto $LCA(u, v) = a$.

5.2 Por que LCA resuelve caminos

En un arbol hay un unico camino simple entre dos nodos. Si quieres responder algo sobre el camino entre u y v , el LCA es el punto donde ese camino se parte:

$$u \rightarrow LCA(u, v) \rightarrow v.$$

Por eso LCA aparece en distancia, maximo peso en camino, minimo peso en camino, numero de aristas, ancestros y problemas tipo "Imperial Roads".

5.3 Solucion simple: binary lifting

Preprocesas:

$$up[v][j] = \text{ancestro } 2^j \text{ de } v.$$

Esto es una tabla de saltos. La columna 0 guarda saltos de una arista; la columna 1 guarda saltos de dos aristas; la columna 2 guarda saltos de cuatro aristas; y asi sucesivamente.

La construccion se basa en:

$$up[v][0] = parent[v],$$

$$up[v][j] = up[up[v][j-1]][j-1].$$

Se lee asi: para subir 2^j niveles desde v , primero subes 2^{j-1} niveles y desde ese nodo vuelves a subir 2^{j-1} niveles.

Para responder:

1. Si un nodo esta mas profundo que el otro, lo subes hasta que ambos tengan la misma profundidad.
2. Si despues de igualarlos son el mismo nodo, ese nodo era ancestro del otro y ya es la respuesta.
3. Si son distintos, subes ambos al mismo tiempo desde potencias grandes a pequenas.
4. Cuando $up[u][j] \neq up[v][j]$, puedes subir ambos 2^j niveles sin pasar por encima del LCA.
5. Al final, u y v quedan justo debajo del LCA; entonces $parent[u]$ es la respuesta.

Complejidad:

$$O(n \log n) \text{ preproceso, } O(\log n) \text{ query.}$$

5.4 Correctitud de binary lifting

La primera fase no cambia el LCA: si un nodo esta mas abajo, los vertices que estan estrictamente debajo de la profundidad del otro nodo no pueden ser ancestros comunes. Por eso se pueden eliminar subiendo.

La segunda fase mantiene el invariante de que u y v siguen debajo del LCA, pero en ramas distintas. Se prueban saltos grandes primero para acercarse lo mas posible al LCA. Si un salto llevara a ambos al mismo ancestro, ese salto alcanza o pasa el LCA, asi que no se toma. Cuando ya no hay salto valido, los padres de ambos coinciden y ese padre es el LCA.

5.5 Binary lifting para subir k niveles

La misma tabla resuelve **Level Ancestor**. Para subir k niveles desde v , escribes k como suma de potencias de dos. Por ejemplo:

$$13 = 8 + 4 + 1.$$

Entonces haces:

$$v \leftarrow up[v][3], \quad v \leftarrow up[v][2], \quad v \leftarrow up[v][0].$$

Cada bit prendido de k representa un salto. Esto cuesta $O(\log n)$.

5.6 Reduccion a RMQ con Euler tour

Haces un Euler tour del arbol. Guardas:

- E : secuencia de nodos visitados;
- D : profundidad de cada visita;
- $first[v]$: primera posicion de v en E .

Entonces:

$$LCA(u, v) = E[\arg \min D[l..r]]$$

donde:

$$l = \min(first[u], first[v]), \quad r = \max(first[u], first[v]).$$

5.7 Propiedad ± 1

En el Euler tour, cada paso sube o baja una arista. Por eso:

$$|D[i] - D[i - 1]| = 1.$$

El problema LCA se reduce a ± 1 RMQ, que puede resolverse en $O(n)$ espacio y $O(1)$ query con macro/micro.

Que significa “ ± 1 RMQ”: es un RMQ sobre un arreglo donde dos valores consecutivos difieren exactamente en uno. En el Euler tour, la profundidad sube o baja una arista en cada paso, por eso aparece esta propiedad. Esa restriccion permite usar bloques pequenos y precomputar respuestas segun la forma del bloque.

5.8 Correctitud

Entre la primera aparicion de u y la primera aparicion de v en el Euler tour, el recorrido pasa por todos los nodos necesarios para ir de uno al otro en el DFS. El nodo de profundidad minima en ese segmento es exactamente el ancestro comun mas bajo. Cualquier ancestro comun esta por encima; el mas bajo es el de menor profundidad que aparece entre ambos.

5.9 Hint de clasificacion

Si ves:

- distancia en arbol;
- camino entre dos nodos;
- ancestro comun;
- max/min edge on path con arbol fijo;

probablemente necesitas LCA. Para distancias:

$$\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2\text{depth}[LCA(u, v)].$$

5.10 Como decidir entre las dos soluciones de LCA

- Si quieres una solución sencilla de explicar: binary lifting.
- Si el problema conecta explícitamente LCA con RMQ: Euler tour + RMQ.
- Si piden query $O(1)$ con espacio lineal: Euler tour + ± 1 RMQ macro/micro.
- Si hay pesos en aristas y consultas de máximo/mínimo en camino: binary lifting puede guardar también el máximo/mínimo de cada salto.

6 Level Ancestor

6.1 Definicion

La consulta **Level Ancestor** pide: dado un nodo v y un nivel/profundidad d , encontrar el ancestro de v que esta a profundidad d . Equivalentemente, subir k aristas desde v .

6.2 Soluciones vistas en slides

El slide compara:

Metodo	Query	Idea
Tabla completa	$O(1)$	demasiado espacio, $O(n^2)$
Jump pointers	$O(\log n)$	binary lifting
Longest path decomposition	$O(\sqrt{n})$	caminos raiz-hoja
Ladder decomposition	$O(\log n)$	caminos extendidos
Macro/micro Dietz	$O(1)$	leaf trimming + Four Russians

6.3 Que debes saber para examen

Domina binary lifting. Si preguntan optimalidad, menciona que existen versiones lineales con query constante, pero no intentes reconstruir todos los detalles si no es necesario.

6.4 Hint de clasificacion

Si el problema pregunta:

- “k-th ancestor”;
- “subir k niveles”;
- “ancestro en profundidad h”;

usa Level Ancestor/binary lifting.

7 Strings: notacion basica

7.1 Conceptos

Los slides definen:

- Alfabeto Σ .
- Cadena S de longitud $|S|$.
- Subcadena $S[i, j]$.
- Prefijo $\text{Pref}(S, i) = S[0, i]$.
- Sufijo $\text{Suf}(S, i) = S[i, |S| - 1]$.
- LCP: prefijo comun mas largo entre dos cadenas.
- Sentinela $\$$: caracter menor que todos que aparece al final.

7.2 Subcadena, subsecuencia, prefijo y sufijo

Estos terminos se parecen, pero no significan lo mismo:

- **Subcadena** o substring: bloque contiguo. En **banana**, **ana** es subcadena.
- **Subsecuencia**: caracteres en orden, pero no necesariamente contiguos. En **banana**, **bnn** es subsecuencia, no subcadena.
- **Prefijo**: subcadena que empieza en la posicion 0.
- **Sufijo**: subcadena que termina en la ultima posicion.

Suffix array trabaja con sufijos, pero sirve para subcadenas porque toda subcadena $S[l..r]$ es prefijo del sufijo $S[l..]$.

7.3 Por que importa el sentinel

El sentinel evita casos molestos donde un sufijo es prefijo de otro. Si $\$$ es menor que todo, entonces las comparaciones lexicograficas son totales y limpias.

7.4 Orden lexicografico

Dos cadenas se comparan por el primer caracter donde difieren. Si una termina antes y es prefijo propio de la otra, la mas corta es menor. Esta definicion es la base de suffix array.

7.5 Idea clave para problemas

Si una subcadena P aparece en S empezando en la posicion i , entonces el sufijo $S[i..]$ empieza con P . Por eso buscar una subcadena se transforma en buscar sufijos que tienen cierto prefijo.

7.6 Hint de clasificacion

Si el problema habla de:

- patron;
- subcadena;
- sufijo;
- prefijo comun;
- orden lexicografico;
- substring repetida;

debes pensar inmediatamente en suffix array, LCP, KMP/Z, o Aho-Corasick.

8 Suffix array

8.1 Definicion

El suffix array de S es una permutacion A de posiciones:

$$\text{Suf}(S, A_i) < \text{Suf}(S, A_{i+1})$$

para todo i . Guarda exactamente n enteros, por eso es mas compacto que un suffix tree.

Mas textualmente: escribes todos los sufijos de S , los ordenas como palabras de diccionario y guardas solo el indice donde empieza cada sufijo. No guardas las cadenas completas.

Ejemplo:

$$S = \text{banana\$}.$$

Los sufijos son:

0		banana\$
1		anana\$
2		nana\$
3		ana\$
4		na\$
5		a\$
6		\$

Ordenados:

6		\$
5		a\$
3		ana\$
1		anana\$
0		banana\$
4		na\$
2		nana\$

Entonces:

$$\text{SA} = [6, 5, 3, 1, 0, 4, 2].$$

8.2 Por que sirve

Ordenar sufijos convierte busqueda de patrones en busqueda binaria:

- Un patron P aparece donde algun sufijo empieza con P .
- Todos esos sufijos estan consecutivos en el suffix array.
- Por tanto, las ocurrencias son un intervalo en SA.

La frase “intervalo en el suffix array” es clave. Si todos los sufijos que empiezan con **ana** estan ordenados lexicograficamente, no pueden quedar separados por un sufijo que no empieza con **ana**. Todos comparten el mismo prefijo, asi que forman un bloque contiguo.

8.3 Rank inverso

Ademas de SA, normalmente se guarda:

$$\text{rank}[i] = \text{posicion del sufijo } S[i..] \text{ dentro de SA}.$$

Si $\text{SA} = [6, 5, 3, 1, 0, 4, 2]$, entonces $\text{rank}[3] = 2$ porque el sufijo que empieza en 3 aparece en la posicion 2 del suffix array. Esto permite comparar sufijos en $O(1)$: menor rank significa menor sufijo.

8.4 Aplicaciones de examen

- Buscar si P aparece en S .
- Contar ocurrencias de P .
- Listar posiciones donde aparece P .
- Contar subcadenas distintas.
- Longest common substring.
- Comparar/ordenar substrings.
- Encontrar repeticiones usando LCP.

8.5 Hint de clasificacion

Si el texto es fijo y hay muchas consultas de patrones, usa suffix array. Si hay muchos patrones y quieres procesarlos todos simultaneamente, Aho-Corasick puede ser mejor. Si necesitas ordenar substrings, suffix array + LCP + RMQ.

9 Manber–Myers: construccion $O(n \log n)$

9.1 Idea central

Manber–Myers ordena sufijos por prefijos de longitud creciente:

$$1, 2, 4, 8, \dots$$

Mantiene $rank[i]$: clase lexicografica del prefijo de longitud actual que empieza en i .

Al inicio solo sabes ordenar por el primer caracter. Despues usas esa informacion para ordenar por dos caracteres, luego por cuatro, luego por ocho. Cuando la longitud considerada supera n , ordenar esos prefijos equivale a ordenar los sufijos completos.

9.2 Que es una clase o rank temporal

Dos posiciones tienen el mismo rank temporal si sus prefijos de la longitud actual son iguales. Por ejemplo, en la primera ronda, todas las posiciones con caracter **a** tienen el mismo rank, todas las posiciones con **b** otro rank, etc. En rondas posteriores el rank representa bloques mas largos.

9.3 Paso inductivo

Si ya conoces ranks para longitud 2^k , entonces el prefijo de longitud 2^{k+1} de i queda descrito por:

$$(rank[i], rank[i + 2^k]).$$

Ordenar estos pares ordena por longitud doble.

9.4 Pseudocodigo teorico

1. Agrega sentinela.
2. Ordena posiciones por caracter y asigna ranks iniciales.
3. Para $len = 1, 2, 4, \dots$:
 - (a) Ordena posiciones por $(rank[i], rank[i + len])$.
 - (b) Reasigna nuevos ranks.
4. Cuando $len \geq n$, el orden es el suffix array.

9.5 Optimizacion del slide

El slide explica que se puede evitar ordenar por el segundo componente desde cero. Como las posiciones ya estan ordenadas por la segunda mitad, se desplazan indices y se usa counting sort estable por el primer rank. Esto mantiene $O(n)$ por ronda.

9.6 Correctitud

Por induccion. En la ronda k , $rank[i]$ representa correctamente el orden del prefijo de longitud 2^k . Dos prefijos de longitud 2^{k+1} se comparan por su primera mitad; si empatan, por su segunda mitad. Eso es exactamente comparar el par de ranks.

9.7 Complejidad

$O(\log n)$ rondas. Cada una $O(n)$ con counting/radix sort. Total:

$$O(n \log n)$$

tiempo y $O(n)$ o $O(n \log n)$ espacio segun si guardas todos los niveles.

10 LCP array y RMQ sobre LCP

10.1 Definición

Para suffix array SA:

$$\text{LCP}[i] = \text{lcp}(S[\text{SA}[i-1]..], S[\text{SA}[i]..]).$$

Mide cuanto comparten sufijos vecinos.

Ejemplo con **banana\$**: los sufijos vecinos **ana\$** y **anana\$** tienen LCP 3, porque comparten **ana**. Los sufijos **na\$** y **nana\$** tienen LCP 2, porque comparten **na**.

10.2 Por que solo vecinos

El LCP array guarda LCP entre vecinos porque el orden lexicografico junta sufijos parecidos. Si quieres detectar repeticiones largas, normalmente aparecen como LCP grande entre sufijos cercanos. Si quieres el LCP de sufijos no vecinos, lo obtienes con RMQ sobre los LCP intermedios.

10.3 LCP arbitrario entre dos sufijos

Si los sufijos que empiezan en i y j estan en posiciones a y b del suffix array, con $a < b$, entonces:

$$\text{lcp}(i, j) = \min(\text{LCP}[a+1], \dots, \text{LCP}[b]).$$

Esto es una RMQ sobre el arreglo LCP.

Textualmente: para saber cuanto comparten dos sufijos lejanos en el suffix array, miras el tramo entre ellos. El prefijo comun de todos los sufijos de ese tramo esta limitado por el menor LCP entre vecinos. Por eso aparece un minimo.

10.4 Correctitud

Entre dos sufijos en orden lexicografico, todos los sufijos intermedios comparten al menos el prefijo comun minimo del intervalo. El primer punto donde alguno difiere queda capturado por el minimo LCP entre vecinos. Por eso el LCP de dos sufijos cualesquiera es el minimo LCP en el rango entre sus posiciones.

10.5 Usos

- Comparar substrings en $O(1)$.
- Encontrar repeticiones.
- Longest common substring.
- Ordenar substrings.
- Resolver problemas donde muchas queries piden igualdad de substrings.

10.6 Plantilla de examen para SA + LCP

Cuando uses suffix array y LCP, escribe:

1. Construyo SA para ordenar todos los sufijos.
2. Construyo *rank* para saber la posicion de cada sufijo en SA.
3. Construyo LCP entre sufijos vecinos.
4. Si necesito LCP arbitrario, preproceso RMQ sobre LCP.
5. La correctitud viene de que el orden lexicografico agrupa sufijos con prefijos comunes.
6. La complejidad depende de la construccion de SA: $O(n \log n)$ con Manber-Myers o $O(n)$ con DC3/SA-IS.

10.7 Hint de clasificacion

Si ves comparaciones repetidas de substrings, no compares caracter por caracter. Preprocesa suffix array, rank, LCP y RMQ.

11 Pattern matching

11.1 Dos escenarios de los slides

Los slides separan:

- Patron conocido, texto por conocer: KMP, Z, Rabin-Karp, Aho-Corasick.
- Texto conocido, patron por conocer: suffix array, suffix tree, suffix automaton.

11.2 Texto fijo + patrones

Construye suffix array de S . Para patron P , busca el intervalo de sufijos que empiezan con P . Dos busquedas binarias bastan.

11.3 Muchos patrones

Si todos los patrones se conocen antes y quieres recorrer el texto una vez, Aho-Corasick es natural. Si el examen esta en el bloque de suffix array, explica la solucion con suffix array.

11.4 Correctitud del intervalo

Todos los sufijos con prefijo P forman un bloque lexicografico. Si un sufijo sin prefijo P estuviera dentro, diferiria en algun caracter y romperia el orden del bloque. Por eso el bloque encontrado por binary search contiene exactamente las ocurrencias.

11.5 Complejidad

Simple:

$$O(n \log n) \text{ construir, } O(|P| \log n) \text{ por patron.}$$

Con LCP optimizado se puede mejorar, pero en papel la idea del intervalo es lo mas importante.

12 Contar subcadenas distintas

12.1 Formula

Un string de longitud n tiene:

$$\frac{n(n+1)}{2}$$

subcadenas contando repeticiones. Para contar distintas:

$$\#distinct = \frac{n(n+1)}{2} - \sum_i \text{LCP}[i].$$

12.2 Por que funciona

Cada sufijo aporta todos sus prefijos como subcadenas. Si procesas sufijos en orden de suffix array, el sufijo actual comparte $\text{LCP}[i]$ prefijos con el anterior. Esos prefijos ya fueron contados. Por tanto, el sufijo aporta:

$$(n - \text{SA}[i]) - \text{LCP}[i]$$

nuevas subcadenas.

12.3 Hint de clasificacion

Si el problema pide:

- numero de substrings distintas;
- cuantas subcadenas diferentes tiene S ;
- contar repetidas vs unicas;

usa suffix array + LCP y esta formula.

12.4 Complejidad

Construir SA + LCP domina. Luego sumar LCP es $O(n)$.

13 Longest common substring

13.1 Problema

Dadas cadenas A y B , encontrar la subcadena mas larga que aparece en ambas.

13.2 Algoritmo

Concatena:

$$T = A\#B\$$$

con separadores que no aparecen en las cadenas. Construye SA y LCP de T . Recorre pares vecinos en SA. Si un sufijo viene de A y el otro de B , actualiza respuesta con su LCP.

13.3 Correctitud

Si una subcadena X aparece en ambas cadenas, entonces hay un sufijo de A y uno de B que tienen prefijo X . Todos los sufijos con prefijo X estan juntos en SA. Dentro de ese bloque debe haber algun par vecino de origen distinto. Su LCP es al menos $|X|$.

13.4 Complejidad

$O(n \log n)$ con Manber–Myers o $O(n)$ con DC3/SA-IS para construir. Recorrido final $O(n)$.

14 Ordenar y comparar subcadenas

14.1 Problema

Muchas veces te dan consultas con substrings $S[l, r]$ y debes compararlas, ordenarlas o decidir igualdad.

14.2 Estructura

Preprocesa:

- SA;
- rank inverso de cada sufijo;
- LCP;
- RMQ sobre LCP.

14.3 Comparacion

Para comparar $S[l_1, r_1]$ y $S[l_2, r_2]$:

1. Calcula $x = \text{lcp}(l_1, l_2)$ con RMQ sobre LCP.
2. Si x cubre la longitud menor, gana la substring mas corta.
3. Si no, compara $S[l_1 + x]$ y $S[l_2 + x]$.

14.4 Complejidad

Preprocesamiento $O(n \log n)$ con sparse table sobre LCP. Cada comparacion $O(1)$. Ordenar q substrings cuesta $O(q \log q)$ comparaciones.

15 DC3

15.1 Idea

DC3 construye suffix array en tiempo lineal. Divide posiciones por residuo modulo 3:

$$S_0 = \{i : i \equiv 0\}, \quad S_1 = \{i : i \equiv 1\}, \quad S_2 = \{i : i \equiv 2\}.$$

Primero ordena recursivamente sufijos de $S_1 \cup S_2$. Luego deduce S_0 y hace merge.

15.2 Por que sirve

El teorema del slide dice: si sabes el orden de sufijos en S_1 y S_2 , puedes deducir el orden de todos. Para comparar un sufijo de S_0 con otro, usas caracteres iniciales y ranks ya conocidos.

15.3 Pasos

1. Formar ternas de caracteres para posiciones en S_1 y S_2 .
2. Ordenar ternas con radix sort y asignar ranks.
3. Si hay empates, resolver recursivamente la cadena de ranks.
4. Ordenar S_0 usando $(S[i], rank[i + 1])$.
5. Mezclar linealmente S_0 con $S_1 \cup S_2$.

15.4 Complejidad

La recurrencia:

$$T(n) = T(2n/3) + O(n)$$

da:

$$T(n) = O(n).$$

15.5 Hint de examen

No intentes memorizar implementacion. Aprende:

- particion modulo 3;
- ordenar $2n/3$ posiciones recursivamente;
- usar ranks para comparar en $O(1)$;
- recurrencia lineal.

16 SA-IS y clasificacion S/L

16.1 Idea

SA-IS clasifica sufijos en tipo S y tipo L:

- tipo S si el sufijo actual es menor que el siguiente;
- tipo L si es mayor que el siguiente;
- LMS si hay transicion de L a S.

16.2 Teorema importante

Si sabes el orden relativo de los sufijos LMS, puedes inducir el orden de todos los sufijos. Por eso SA-IS:

1. clasifica S/L;
2. ordena LMS;
3. induce L de izquierda a derecha;
4. induce S de derecha a izquierda;
5. si hace falta, recurre sobre nombres de LMS substrings.

16.3 Por que es lineal

Cada fase de induccion escanea buckets una cantidad constante de veces. El problema recursivo tiene tamano a lo mucho el numero de LMS, que es a lo mucho $n/2$. Por tanto el total es lineal.

16.4 Hint de examen

SA-IS es mas conceptual que implementable a papel. Si preguntan, escribe: clasificacion S/L, LMS, induced sorting, recursion sobre LMS names, y complejidad $O(n)$.

17 Bordes, KMP y periodicidad

17.1 Bordes

Un borde de S es una cadena que es prefijo y sufijo. La funcion prefijo de KMP:

$$\pi[i] = \text{longitud del mayor borde de } S[0..i].$$

Los bordes de todo S se obtienen siguiendo:

$$k = \pi[n - 1], \quad k = \pi[k - 1], \quad \dots$$

17.2 Periodicidad

S tiene periodo p si:

$$S[i] = S[i + p]$$

cuando ambas posiciones existen. Para substrings, puedes verificar igualdad de zonas con LCP:

$$\text{lcp}(l, l + p) \geq \text{len} - p.$$

17.3 Hint de clasificacion

Si dice:

- borde;
- prefijo que tambien es sufijo;
- periodo;
- repeticion exacta;

piensa primero en KMP/Z. Si hay muchas consultas sobre substrings arbitrarias, combina con suffix array + LCP.

18 Problemas tipo examen y estructura a usar

Si el problema dice	Usa	Justificacion
Minimo en rango estatico	Sparse table / RMQ	Datos no cambian
LCA de muchos pares	Euler tour + RMQ o binary lifting	Arbol fijo
Distancia entre nodos	LCA	Formula con profundidades
Forzar arista en MST	MST + max edge path	Propiedad del ciclo
Buscar patron en texto fijo	Suffix array	Ocurrencias son intervalo
Contar subcadenas distintas	SA + LCP	Duplicados medidos por LCP
Longest common substring	SA de $A\#B$ + LCP	Pares vecinos de origen distinto
Ordenar substrings	SA + LCP + RMQ	Comparacion en $O(1)$
Bordes/periodos	KMP/Z, LCP si hay queries	Prefijo-sufijo/repeticion

19 Mapa de slides cubiertos

eda_slides_08_rm.pdf

- Pages 1–2: Static Trees overview.
- Pages 3–6: objetivos: RMQ, LCA, Level Ancestor con preproceso lineal y query constante.
- Pages 11–12: definicion formal de RMQ.
- Pages 13–16: Cartesian trees y reduccion LCA a ± 1 RMQ via Euler tour.
- Pages 17–18: solucion RMQ con macro/micro bloques.
- Pages 19–20: Level Ancestor y comparacion de tecnicas.
- Pages 22–24: resumen y relacion con segment tree/Fenwick.

Estructuras_de_Datos_Avanzadas____Strings_I.pdf

- Pages 2–5: objetivos: RMQ, LCA y pattern matching.
- Pages 6–11: RMQ estatico y soluciones de preprocesamiento/query.
- Pages 12–17: LCA con binary lifting y reducciones.
- Pages 19–24: definicion de pattern matching.
- Pages 25–30: escenarios: patron fijo vs texto fijo; KMP/Z/Rabin-Karp/Aho-Corasick vs suffix array/tree/automaton.
- Pages 31–34: resumen de sesion.

Estructuras_de_Datos_Avanzadas____Strings_II.pdf

- Pages 2–5: strings y suffix array.
- Pages 12–18: definicion de suffix array y algoritmos SACA.
- Pages 20–44: DC3: particion modulo 3, ranks, recursion, merge, recurrencia lineal.
- Pages 45–60: SA-IS: tipos S/L, LMS, induced sorting.
- Pages 61–80: resumen/ejemplos y construccion lineal conceptual.

CS3014____Strings.pdf

- Page 1: notacion formal de strings, prefijos, sufijos, LCP, sentinela.
- Pages 2–7: suffix array y Manber–Myers.
- Pages 8–10: DC3 con pseudocodigo y merge.
- Pages 10–18: clasificacion de sufijos, induced sorting, Ko-Aluru/SAIS.
- Pages 19–23: referencias, cierre y detalles de implementacion teorica.

20 Apendice: mapa pagina por pagina de los slides

Este apendice lista cada pagina/slide de la carpeta fuente y su contenido principal. Varias paginas son animaciones repetidas del mismo slide; se conservan para trazabilidad.

CS3014____Strings.pdf (23 paginas)

Pag. 1

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Strings 31 de mayo de 2026 Prof. V'ctor Racs' o Galv' an Oyola 1. Notaci' on Sea Sigma un conjunto ordenado de s' mbolos (a partir de ahora,caracteres), el cual llamaremos alfabeto. Diremos que una secuencia finitaSque consiste de caracteres de Sigma es unacadenasobre el alfabeto Sigma. Por convenci' on, denot...

Pag. 2

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 SyTdifieren en una posici' on. Seapla menor posici' on en la que difieren, entonces $S[p] < T[p]$. Por convenci' on, usaremos comparaci' on lexicogr' afica al comparar dos cadenas. 2. Suffix Array El suffix array es una estructura de indexing cuya caracter' stica principal es la poca cantidad de memoria...

Pag. 3

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 1:Construir-Mapeos(S, n) $1k \leftarrow 1 + \text{floor}(\log_2 n)$; $2\text{Seamapeo}[k][n]$ un arreglo de enteros ; $3p \leftarrow \{0, \dots, n-1\}$; 4OrdenarpporS i creciente ; $5\text{Asignarmapeo}[0]$ de acuerdo ap; $6\text{parad} \leftarrow 1$ ak-1hacer $7v \leftarrow \{(\text{mapeo}[d-1][i], \text{mapeo}[d-1][i + 2^{d-1}]) : 0 \leq i < n - 2^d\}$; 8Filtrarpp para quedarnos con solo losi in $[0, n - 2^d]$; $9O \dots$

Pag. 4

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Nivel $d = 1$ Se filtra el vectorpy mapeamosi $7 > i - 2^{d-1} = i - 1$: (1,4,7,10,8,9,2,3,5,6)- $\rightarrow$ (0,3,6,9,7,8,1,2,4,5). Luego de ordenar de manera estable seg' unmapeo[0], se obtiene: $p = (7,1,4,0,9,8,3,6,2,5)$. El nuevo mapeo es: $i \ 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10$ mapeo[1] $2 \ 1 \ 6 \ 5 \ 1 \ 6 \ 5 \ 0 \ 4 \ 3 \ ?$ Nivel $d = 2$ Se filtrapy mapeamosi $7 \dots$

Pag. 5

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 haremos ... ?o s' ? En realidad es posible extenderSusando $2^m - 1$ caracteres sentinelas \$. Esta ' ultima condici' on es necesaria para representar cadenas que originalmente no estar' an completas (en cuyo caso, la relaci' on cuando una es prefijo de otra se establece correctamente ya que \$ precede a...

Pag. 6

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Iteraci' onlen= 1 El mapeo inicial queda dado por: $i \ 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11$ mapeo $2 \ 1 \ 4 \ 4 \ 1 \ 4 \ 4 \ 1 \ 3 \ 3 \ 1 \ 0$ Iteraci' onlen= 2 Luego de asignar el vectorby second, este queda de la siguiente manera: $i \ 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11$ by second $2 \ 1 \ 4 \ 4 \ 1 \ 4 \ 4 \ 1 \ 3 \ 3 \ 1 \ 0$ Y losheadquedar' an de la siguiente mane...

Pag. 7

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 $i \ 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11$ head $0 \ 1 \ 2 \ 3 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 0$ Luego de ejecutar la l' neas 10-13 del algoritmo, se obtiene: $a = (11,10,7,4,1,0,9,8,6,3,5,2)$. El nuevo mapeo es: $i \ 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11$ mapeo $5 \ 4 \ 11 \ 9 \ 3 \ 10 \ 8 \ 2 \ 7 \ 6 \ 1 \ 0$ Iteraci' onlen= 16 Luego de asignar el vectorby second, este queda de...

Pag. 8

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 $R_k = \text{Suf}(S, k)$ agrupado en grupos de 3 caracteres consecutivos Consideremos que si alg' un grupo contiene menos de 3 caracteres, completaremos dichos caracteres con sentinelas \$. Por ejemplo, para $S = \text{mississippiconn}$ = 11, se dar' a que $R_1 = (\text{iss})(\text{iss})(\text{ipp})(\text{i}\$)$ $R_2 = (\text{ssi})(\text{ssi})(\text{ppi})$ Adem' as, se define $R = R \dots$

Pag. 9

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 i in B_0 y j in B_1 : Comparamos por pares como en B_0 . i in B_0 y j in B_2 : Comparamos por ternas $(S_i, S_{i+1}, \text{rank } S_{i+2})$ y $(S_j, S_{j+1}, \text{rank } S_{j+2})$. Asumiremos que tendremos una funci' on llamadaCompare-DC3(i, j, S, rnk) que permita comparar las posiciones i y j seg' un este criterio (asumiremos que devuelve una respuest...

Pag. 10

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 5:DC3(n, S) 1Sean $B_k = \{0 \leq i < n: i = k(m' \bmod 3)\}$ 2parak= 0,1,2 ; 2sin=1 (m' mod 3) entonces 3B 1 <- B 1 $\cup \{n\}$; 4C<-B 1 $\cup B$ 2 ; 5OrdenarCpor (S[x], S[x+ 1], S[x+ 2]) conRadix-Sort; 6Crearsigma:C7->Z + 0 seg' un el orden deC; 7R<- 2Q i=1 Q x in Bi sigma((S[x], S[x+ 1], S[x+ 2])) ; 8A R <-DC3(R, R) ; 9Searnk[0. . . n]...

Pag. 11

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 6:Classify(n, S) 1SeaT[0. . .(n-1)] un arreglo de tipos ; 2T[n-1]<-S; 3parai<-n-2a0hacer 4siS[i] =S[i]entonces 5T[i]<-T[i+ 1] ; 6en otro caso 7siS[i]<S[i+ 1]entonces 8T[i]<-S; 9en otro caso 10T[i]<-L; 11devolverT; Debido a la definici' on de los tipos, se puede probar el siguiente lema: Lema 7 S...

Pag. 12

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 2. Iteramos de izquierda a derecha (derecha a izquierda) sobre los elementos deA, siS A[i]-1 es de tipoL(S), entonces lo colocamos al frente (final) de su bloque y movemos el puntero correspondiente. Es sencillo notar que la complejidad del algoritmo anterior esO(n), ya que podemos mantener el inicio...

Pag. 13

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 b d e f d f d f b d e d d a f d\$ Tipo S S S L S L S L S L L L S L L S/L Cuadro 1: Tipos de sufijo de la cadena "bdefdfdbdeddaafd\$" Subcadenas S Subcadenas L afd\$ bd de dedda dfb dfd efd d\$ daf dd ed fbde fd fdf Cuadro 2: Subcadenas de la cadena "bdefdfdbdeddaafd\$" Si asoci' aramos cada subcadena a s...

Pag. 14

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Existe una posici' on tal que $X_j = Y_j$ y $X_j < Y_j$. Y(X) es prefijo propio deX(Y). A partir de este enfoque, se cumple el siguiente lema. Lema 13 SeaS ' la cadena compuesta por los mapeos de las subcadenas elegidas deS, entonces para dos sufijosS i yS j con sus posiciones correspondientes enS ' i' yS '...

Pag. 15

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 funciones deben modificarse. Algoritmo 9:Group-By-Character(n, S) 1SeaA[n] un arreglo de enteros ; 2Seahead[n] un arreglo de enteros inicializado en 0 ; 3parai<-0an-1hacer 4head[S[i]]<-head[S[i]] + 1 ; 5sum<-0 ; 6parai<-0an-1hacer 7sum<-sum+head[i] ; 8head[i]<-sum-head[i] ; 9parai<-0an-1hacer 10A[head[S[i]]]...

Pag. 16

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 12:Move-To-Front(R, lptr, L j, l, r) 1parai<-lar-1hacer 2start<-L j[i]-j; 3pos bucket<-R[start] ; 4f ront bucket<-lptr[pos bucket] ; 5sif ront bucket <0entonces 6f ront bucket<- not f ront bucket; 7lptr[pos bucket]<- not (f ront bucket+ 1) ; 8sif ront bucket=pos bucketentonces 9lptr[pos bucket]<- not lptr[p...

Pag. 17

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 posiciones con sufijos de tipo S. Algoritmo 15:Build-Reduced-String-With-S(n, S, T) 1A<-Group-By-Character(n, S) ; 2Dist<-Get-S-Distances(n, S, T) ; 3m<- n-1 m' ax i=0 {Dist[i]}; 4SeanL 1 . . . Lm listas vac' as de enteros ; 5parai<-0an-1hacer 6idx<-A[i]; 7siDist[idx]>0entonces 8Agrega idxal final deL Dis...

Pag. 18

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 16:SAKA(n, S) 1sin= 1entonces 2devolver{0}; 3en otro caso 4T<-Classify(n, S) ; 5cnt L <- |{0<=i < n-1 :T i =L}|; 6cnt S <-n-1-cnt L ; 7sient L < cnt S entonces 8S ' <-Build-Reduced-String-With-L(n, S, T) ; 9siS ' tiene todos sus caracteres diferentesentonces 10CalcularXen funci' on al caracter de...

Pag. 19

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Definici' on 16 (Subcadena LMS) Se define una subcadena LMS como una subcadenaS[i, j] tal que se cumpla alguna de las siguientes condiciones: i < j y tanto Suf(S, i) como Suf(S, j) son sufijos LMS y no existe un ' ndicei < k < j tal que Suf(S, k) sea un sufijo de tipo LMS. S[i, j] = Suf(S, n-1). En la...

Pag. 20

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 17:Compute-Buckets(n , S , end)
1Sea $cnt[0..(|Sigma|-1)]$ un arreglo de enteros inicializado con 0 ; 2para c in $Shacer$ 3 $cnt[c] \leftarrow cnt[c] + 1$
; 4Sea $P[0..(|Sigma|-1)]$ un arreglo de enteros ; 5 $sum \leftarrow 0$; 6para $i \leftarrow 0$ a $|Sigma|-1$ hacer 7si $end = F$ alse entonces
8 $P[i] \leftarrow sum$; 9 $sum \leftarrow sum + cnt[i]$; 10en otro caso 11 $sum \leftarrow sum + cnt[i]$;...

Pag. 21

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Algoritmo 18:Induce-Order(n , S , T , X) 1Sea $A[0..(n-1)]$ un arreglo de enteros inicializado en 0 ; 2 $R \leftarrow \text{Compute-Buckets}(n, S, \text{True})$; 3para $i \leftarrow |X| - 1$ a 0 hacer 4 $c \leftarrow S[X[i]]$; 5 $A[R[c]] \leftarrow X[i]$; 6 $R[c] \leftarrow R[c] - 1$; 7 $L \leftarrow \text{Compute-Buckets}(n, S, \text{False})$; 8para $i \leftarrow 0$ a $n-1$ hacer 9si $A[i] > 0$ and $T[A[i]-1] = L$ entonces 10 $c \leftarrow S[A[i]-1]$; 11 $A[...$

Pag. 22

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 es $O(n)$. El siguiente teorema garantiza la correctitud del algoritmo SAIS. Teorema 23 (Nonget al., 2009) Es posible ordenar los sufijos de S llamando a $\text{Induce-Order}(n, S, T, \text{LMS suffix})$, donde LMS suffix es LMS ordenado por el sufijo de cada posición correspondiente. Para calcular LMS suffix , primero...

Pag. 23

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 [3] Udi Manber and Gene Myers. Suffix arrays: a new method for on-line string searches. *siam Journal on Computing*, 22(5):935-948, 1993. [4] Ge Nong, Sen Zhang, and Wai Hong Chan. Linear suffix array construction by almost pure induced-sorting. In 2009 data compression conference, pages 193-202. IEEE, 20...

Estructuras_de_Datos_Avanzadas____Strings_I.pdf (35 paginas)

Pag. 1

CS3014 Estructuras de Datos Avanzadas CS3014 Estructuras de Datos Avanzadas Laboratorio - Semana 10 - Sesión 1 Victor Racso Galvan Oyola vgalvan@utec.edu.pe 27 de mayo de 2026

Pag. 2

CS3014 Estructuras de Datos Avanzadas ¿Que aprenderemos hoy? -RMQ y LCA -Pattern Matching

Pag. 3

CS3014 Estructuras de Datos Avanzadas ¿Que aprenderemos hoy? -RMQ y LCA -Pattern Matching

Pag. 4

CS3014 Estructuras de Datos Avanzadas ¿Que aprenderemos hoy? -RMQ y LCA -Pattern Matching

Pag. 5

RMQ y LCA

Pag. 6

CS3014 Estructuras de Datos Avanzadas RMQ Range Minimum Query (Codeforces Group - Static RMQ)
Dada una secuencia A de enteros, calcular el valor m tal que $l \leq i \leq r$ $\{A_i\}$ para múltiples consultas. Soluciones
Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta. Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de c...

Pag. 7

CS3014 Estructuras de Datos Avanzadas RMQ Range Minimum Query (Codeforces Group - Static RMQ)
Dada una secuencia A de enteros, calcular el valor m tal que $l \leq i \leq r$ $\{A_i\}$ para múltiples consultas. Soluciones
Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta. Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de c...

Pag. 8

CS3014 Estructuras de Datos Avanzadas RMQ Range Minimum Query (Codeforces Group - Static RMQ)
Dada una secuencia A de enteros, calcular el valor m tal que $l \leq i \leq r$ $\{A_i\}$ para múltiples consultas. Soluciones
Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta. Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de c...

Pag. 9

CS3014 Estructuras de Datos Avanzadas RMQ Range Minimum Query (Codeforces Group - Static RMQ) Dada una secuencia A de enteros, calcular el valor $m = \min_{l \leq i \leq r} \{A_i\}$ para múltiples consultas. Soluciones Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta. Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de c...

Pag. 10

CS3014 Estructuras de Datos Avanzadas RMQ Range Minimum Query (Codeforces Group - Static RMQ) Dada una secuencia A de enteros, calcular el valor $m = \min_{l \leq i \leq r} \{A_i\}$ para múltiples consultas. Soluciones Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta. Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de c...

Pag. 11

CS3014 Estructuras de Datos Avanzadas RMQ Range Minimum Query (Codeforces Group - Static RMQ) Dada una secuencia A de enteros, calcular el valor $m = \min_{l \leq i \leq r} \{A_i\}$ para múltiples consultas. Soluciones Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta. Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de c...

Pag. 12

CS3014 Estructuras de Datos Avanzadas LCA Lowest Common Ancestor Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u, v) con el LCA de u y v . Soluciones Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta. Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta. La implemen...

Pag. 13

CS3014 Estructuras de Datos Avanzadas LCA Lowest Common Ancestor Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u, v) con el LCA de u y v . Soluciones Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta. Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta. La implemen...

Pag. 14

CS3014 Estructuras de Datos Avanzadas LCA Lowest Common Ancestor Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u, v) con el LCA de u y v . Soluciones Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta. Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta. La implemen...

Pag. 15

CS3014 Estructuras de Datos Avanzadas LCA Lowest Common Ancestor Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u, v) con el LCA de u y v . Soluciones Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta. Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta. La implemen...

Pag. 16

CS3014 Estructuras de Datos Avanzadas LCA Lowest Common Ancestor Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u, v) con el LCA de u y v . Soluciones Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta. Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta. La implemen...

Pag. 17

CS3014 Estructuras de Datos Avanzadas LCA Lowest Common Ancestor Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u, v) con el LCA de u y v . Soluciones Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta. Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta. La implemen...

Pag. 18

Pattern Matching

Pag. 19

CS3014 Estructuras de Datos Avanzadas Pattern Matching Definición Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patrón, y se desea saber si P ocurre en T como subcadena. Su

variacion llamada Multiple pattern matching implica tener multiples patrones $P_1, \dots, P_m$. Aplicaciones Busquedas exactas de texto, motores de busqueda...

Pag. 20

CS3014 Estructuras de Datos Avanzadas Pattern Matching Definicion Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patron, y se desea saber si P ocurre en T como subcadena. Su variacion llamada Multiple pattern matching implica tener multiples patrones $P_1, \dots, P_m$. Aplicaciones Busquedas exactas de texto, motores de busqueda...

Pag. 21

CS3014 Estructuras de Datos Avanzadas Pattern Matching Definicion Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patron, y se desea saber si P ocurre en T como subcadena. Su variacion llamada Multiple pattern matching implica tener multiples patrones $P_1, \dots, P_m$. Aplicaciones Busquedas exactas de texto, motores de busqueda...

Pag. 22

CS3014 Estructuras de Datos Avanzadas Pattern Matching Definicion Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patron, y se desea saber si P ocurre en T como subcadena. Su variacion llamada Multiple pattern matching implica tener multiples patrones $P_1, \dots, P_m$. Aplicaciones Busquedas exactas de texto, motores de busqueda...

Pag. 23

CS3014 Estructuras de Datos Avanzadas Pattern Matching Definicion Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patron, y se desea saber si P ocurre en T como subcadena. Su variacion llamada Multiple pattern matching implica tener multiples patrones $P_1, \dots, P_m$. Aplicaciones Busquedas exactas de texto, motores de busqueda...

Pag. 24

CS3014 Estructuras de Datos Avanzadas Pattern Matching Definicion Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patron, y se desea saber si P ocurre en T como subcadena. Su variacion llamada Multiple pattern matching implica tener multiples patrones $P_1, \dots, P_m$. Aplicaciones Busquedas exactas de texto, motores de busqueda...

Pag. 25

CS3014 Estructuras de Datos Avanzadas Escenarios P conocido, T por conocer Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento. Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple). T conocido, P por conocer Consideraremos un escenario en el que se nos da T inicialmente, procesa...

Pag. 26

CS3014 Estructuras de Datos Avanzadas Escenarios P conocido, T por conocer Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento. Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple). T conocido, P por conocer Consideraremos un escenario en el que se nos da T inicialmente, procesa...

Pag. 27

CS3014 Estructuras de Datos Avanzadas Escenarios P conocido, T por conocer Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento. Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple). T conocido, P por conocer Consideraremos un escenario en el que se nos da T inicialmente, procesa...

Pag. 28

CS3014 Estructuras de Datos Avanzadas Escenarios P conocido, T por conocer Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento. Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple). T conocido, P por conocer Consideraremos un escenario en el que se nos da T inicialmente, procesa...

Pag. 29

CS3014 Estructuras de Datos Avanzadas Escenarios P conocido, T por conocer Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento. Algoritmos:

Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple). Tconocido,Ppor conocer Consideraremos un escenario en el que se nos daTinicialmente, procesa...

Pag. 30

CS3014 Estructuras de Datos Avanzadas Escenarios Pconocido,Tpor conocer Consideraremos un escenario en el que se nos daPinicialmente, procesaremos Py luego se nos daTpara hacer el emparejamiento. Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple). Tconocido,Ppor conocer Consideraremos un escenario en el que se nos daTinicialmente, procesa...

Pag. 31

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Implementamos RMQ -Implementamos LCA -Aprendimos de que trata el problema de Pattern Matching

Pag. 32

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Implementamos RMQ -Implementamos LCA -Aprendimos de que trata el problema de Pattern Matching

Pag. 33

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Implementamos RMQ -Implementamos LCA -Aprendimos de que trata el problema de Pattern Matching

Pag. 34

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Implementamos RMQ -Implementamos LCA -Aprendimos de que trata el problema de Pattern Matching

Pag. 35

CS3014 Estructuras de Datos Avanzadas Gracias

Estructuras_de_Datos_Avanzadas_____Strings_II.pdf (81 paginas)

Pag. 1

CS3014 Estructuras de Datos Avanzadas CS3014 Estructuras de Datos Avanzadas Laboratorio - Semana 10 - Sesion 2 Victor Racso Galvan Oyola vgalvan@utec.edu.pe 29 de mayo de 2026

Pag. 2

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Strings -Suffix array

Pag. 3

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Strings -Suffix array

Pag. 4

CS3014 Estructuras de Datos Avanzadas ?Que aprenderemos hoy? -Strings -Suffix array

Pag. 5

Strings

Pag. 6

CS3014 Estructuras de Datos Avanzadas Notacion Notacion basica de strings Una cadena es una secuencia de simbolos llamadoscaracteres, los cuales pertenecen a un conjunto llamadoalfabetoSigma. Eli-esimo caracter de una cadenaSse denota porS[i]y la concatenacion de los caracteres de las posicionesx in [i,j]se denotar porS[i,j]. Convenciones -Se considera comparacionl...

Pag. 7

CS3014 Estructuras de Datos Avanzadas Notacion Notacion basica de strings Una cadena es una secuencia de simbolos llamadoscaracteres, los cuales pertenecen a un conjunto llamadoalfabetoSigma. Eli-esimo caracter de una cadenaSse denota porS[i]y la concatenacion de los caracteres de las posicionesx in [i,j]se denotar porS[i,j]. Convenciones -Se considera comparacionl...

Pag. 8

CS3014 Estructuras de Datos Avanzadas Notacion Notacion basica de strings Una cadena es una secuencia de simbolos llamadoscaracteres, los cuales pertenecen a un conjunto llamadoalfabetoSigma. Eli-esimo caracter de una cadenaSse denota porS[i]y la concatenacion de los caracteres de las posicionesx in [i,j]se denotar porS[i,j]. Convenciones -Se considera comparacionl...

Pag. 9

CS3014 Estructuras de Datos Avanzadas Notacion Notacion basica de strings Una cadena es una secuencia de simbolos llamadoscaracteres, los cuales pertenecen a un conjunto llamadoalfabetoSigma. Eli-esimo caracter de una cadenaSse denota porS[i]y la concatenacion de los caracteres de las posicionesx in [i,j]se denotar porS[i,j]. Convenciones -Se considera comparacionl...

Pag. 10

CS3014 Estructuras de Datos Avanzadas Notacion Notacion basica de strings Una cadena es una secuencia de simbolos llamadoscaracteres, los cuales pertenecen a un conjunto llamadoalfabetoSigma. Eli-esimo caracter de una cadenaSse denota porS[i]y la concatenacion de los caracteres de las posicionesx in [i,j]se denotar porS[i,j]. Convenciones -Se considera comparacionl...

Pag. 11

CS3014 Estructuras de Datos Avanzadas Notacion Notacion basica de strings Una cadena es una secuencia de simbolos llamadoscaracteres, los cuales pertenecen a un conjunto llamadoalfabetoSigma. Eli-esimo caracter de una cadenaSse denota porS[i]y la concatenacion de los caracteres de las posicionesx in [i,j]se denotar porS[i,j]. Convenciones -Se considera comparacionl...

Pag. 12

Suffix array

Pag. 13

CS3014 Estructuras de Datos Avanzadas Suffix array Definicion El suffix array de una cadenaScon longitudnes una permutacionAde sus posiciones de manera que siiva antes quejenA, entonces $S[i,n-1] < S[j,n-1]$. Denotaremos porSuf(S,i)al sufijo deSque comienza en la posicioni, es decir, $S[i,n-1]$. Setup basico En primer lugar, para cualquier cadenaSque tengamos como...

Pag. 14

CS3014 Estructuras de Datos Avanzadas Suffix array Definicion El suffix array de una cadenaScon longitudnes una permutacionAde sus posiciones de manera que siiva antes quejenA, entonces $S[i,n-1] < S[j,n-1]$. Denotaremos porSuf(S,i)al sufijo deSque comienza en la posicioni, es decir, $S[i,n-1]$. Setup basico En primer lugar, para cualquier cadenaSque tengamos como...

Pag. 15

CS3014 Estructuras de Datos Avanzadas Suffix array Definicion El suffix array de una cadenaScon longitudnes una permutacionAde sus posiciones de manera que siiva antes quejenA, entonces $S[i,n-1] < S[j,n-1]$. Denotaremos porSuf(S,i)al sufijo deSque comienza en la posicioni, es decir, $S[i,n-1]$. Setup basico En primer lugar, para cualquier cadenaSque tengamos como...

Pag. 16

CS3014 Estructuras de Datos Avanzadas Suffix array Definicion El suffix array de una cadenaScon longitudnes una permutacionAde sus posiciones de manera que siiva antes quejenA, entonces $S[i,n-1] < S[j,n-1]$. Denotaremos porSuf(S,i)al sufijo deSque comienza en la posicioni, es decir, $S[i,n-1]$. Setup basico En primer lugar, para cualquier cadenaSque tengamos como...

Pag. 17

CS3014 Estructuras de Datos Avanzadas Suffix array Definicion El suffix array de una cadenaScon longitudnes una permutacionAde sus posiciones de manera que siiva antes quejenA, entonces $S[i,n-1] < S[j,n-1]$. Denotaremos porSuf(S,i)al sufijo deSque comienza en la posicioni, es decir, $S[i,n-1]$. Setup basico En primer lugar, para cualquier cadenaSque tengamos como...

Pag. 18

CS3014 Estructuras de Datos Avanzadas Construcccion del suffix array La siguiente tabla muestra algunos algoritmos de construccion de suffix array (SACA) importantes historicamente. Autor(es) Algoritmo Complejidad Idea Manber & Myers Sin nombre $O(n \log n)$ Doubling Itoh & Tanaka Sin nombre Superlineal Ordenamiento inducido Ko & Aluru SAKA $O(n)$ Ordenamiento induc...

Pag. 19

CS3014 Estructuras de Datos Avanzadas Construcccion del suffix array La siguiente tabla muestra algunos algoritmos de construccion de suffix array (SACA) importantes historicamente. Autor(es) Algoritmo Complejidad

Idea Manber & Myers Sin nombre $O(n \log n)$ Doubling Itoh & Tanaka Sin nombre Superlineal Ordenamiento inducido Ko & Aluru SAKA $O(n)$ Ordenamiento induc...

Pag. 20

CS3014 Estructuras de Datos Avanzadas DC3 (Karkkainen et. al.) Idea Particionamos las posiciones segun su residuo respecto a 3. Denotamos los tres conjuntos $S_0 = \{0 \leq i \leq n-1 : i \equiv 3k, \text{para algunk in } Z\}$ $S_1 = \{0 \leq i \leq n-1 : i \equiv 3k+1, \text{para algunk in } Z\}$ $S_2 = \{0 \leq i \leq n-1 : i \equiv 3k+2, \text{para algunk in } Z\}$ Teorema (Karkkainen et. al.) Si se sabe el orden de los sufijos en los elementos de S_1 y S_2 , entonces...

Pag. 21

CS3014 Estructuras de Datos Avanzadas DC3 (Karkkainen et. al.) Idea Particionamos las posiciones segun su residuo respecto a 3. Denotamos los tres conjuntos $S_0 = \{0 \leq i \leq n-1 : i \equiv 3k, \text{para algunk in } Z\}$ $S_1 = \{0 \leq i \leq n-1 : i \equiv 3k+1, \text{para algunk in } Z\}$ $S_2 = \{0 \leq i \leq n-1 : i \equiv 3k+2, \text{para algunk in } Z\}$ Teorema (Karkkainen et. al.) Si se sabe el orden de los sufijos en los elementos de S_1 y S_2 , entonces...

Pag. 22

CS3014 Estructuras de Datos Avanzadas DC3 (Karkkainen et. al.) Idea Particionamos las posiciones segun su residuo respecto a 3. Denotamos los tres conjuntos $S_0 = \{0 \leq i \leq n-1 : i \equiv 3k, \text{para algunk in } Z\}$ $S_1 = \{0 \leq i \leq n-1 : i \equiv 3k+1, \text{para algunk in } Z\}$ $S_2 = \{0 \leq i \leq n-1 : i \equiv 3k+2, \text{para algunk in } Z\}$ Teorema (Karkkainen et. al.) Si se sabe el orden de los sufijos en los elementos de S_1 y S_2 , entonces...

Pag. 23

CS3014 Estructuras de Datos Avanzadas DC3 (Karkkainen et. al.) Idea Particionamos las posiciones segun su residuo respecto a 3. Denotamos los tres conjuntos $S_0 = \{0 \leq i \leq n-1 : i \equiv 3k, \text{para algunk in } Z\}$ $S_1 = \{0 \leq i \leq n-1 : i \equiv 3k+1, \text{para algunk in } Z\}$ $S_2 = \{0 \leq i \leq n-1 : i \equiv 3k+2, \text{para algunk in } Z\}$ Teorema (Karkkainen et. al.) Si se sabe el orden de los sufijos en los elementos de S_1 y S_2 , entonces...

Pag. 24

CS3014 Estructuras de Datos Avanzadas DC3 (Karkkainen et. al.) Idea Particionamos las posiciones segun su residuo respecto a 3. Denotamos los tres conjuntos $S_0 = \{0 \leq i \leq n-1 : i \equiv 3k, \text{para algunk in } Z\}$ $S_1 = \{0 \leq i \leq n-1 : i \equiv 3k+1, \text{para algunk in } Z\}$ $S_2 = \{0 \leq i \leq n-1 : i \equiv 3k+2, \text{para algunk in } Z\}$ Teorema (Karkkainen et. al.) Si se sabe el orden de los sufijos en los elementos de S_1 y S_2 , entonces...

Pag. 25

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Para S_1 y S_2 , vamos a tomar los bloques de 3 elementos en adelante desde cada posicion. Ya que podrian faltar elementos, se hace padding con sentinelas \$. Por ejemplo, para "banana\$", se tendrian los siguientes bloques: $S_1 \rightarrow (ana)(na\$)$ $S_2 \rightarrow (nan)(a\$\$)$ Notemos que es posible mapear cada uno de estos bloq...

Pag. 26

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Para S_1 y S_2 , vamos a tomar los bloques de 3 elementos en adelante desde cada posicion. Ya que podrian faltar elementos, se hace padding con sentinelas \$. Por ejemplo, para "banana\$", se tendrian los siguientes bloques: $S_1 \rightarrow (ana)(na\$)$ $S_2 \rightarrow (nan)(a\$\$)$ Notemos que es posible mapear cada uno de estos bloq...

Pag. 27

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Para S_1 y S_2 , vamos a tomar los bloques de 3 elementos en adelante desde cada posicion. Ya que podrian faltar elementos, se hace padding con sentinelas \$. Por ejemplo, para "banana\$", se tendrian los siguientes bloques: $S_1 \rightarrow (ana)(na\$)$ $S_2 \rightarrow (nan)(a\$\$)$ Notemos que es posible mapear cada uno de estos bloq...

Pag. 28

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Para S_1 y S_2 , vamos a tomar los bloques de 3 elementos en adelante desde cada posicion. Ya que podrian faltar elementos, se hace padding con sentinelas \$. Por ejemplo, para "banana\$", se tendrian los siguientes bloques: $S_1 \rightarrow (ana)(na\$)$ $S_2 \rightarrow (nan)(a\$\$)$ Notemos que es posible mapear cada uno de estos bloq...

Pag. 29

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y ahora que hacemos? Podemos reemplazar cada bloque por su respectivo rank y se terminan creando dos cadenas nuevas. $S_1 \rightarrow R_1 = 12$ $S_2 \rightarrow R_2 = 30$ Si concatenamos

estas dos cadenas nuevas, tendremos una cadena $R=R_1R_2$ de longitud $2n+3$. Aplicando recursion Podemos usar recursion para obtener el suffix array de la cadena R , de es...

Pag. 30

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y ahora que hacemos? Podemosreemplazarcadabloqueporsurespectivorankyseterminancreando dos cadenas nuevas. $S_1 \rightarrow R_1 = 12$ $S_2 \rightarrow R_2 = 30$ Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R=R_1R_2$ de longitud $2n+3$. Aplicando recursion Podemos usar recursion para obtener el suffix array de la cadena R , de es...

Pag. 31

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y ahora que hacemos? Podemosreemplazarcadabloqueporsurespectivorankyseterminancreando dos cadenas nuevas. $S_1 \rightarrow R_1 = 12$ $S_2 \rightarrow R_2 = 30$ Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R=R_1R_2$ de longitud $2n+3$. Aplicando recursion Podemos usar recursion para obtener el suffix array de la cadena R , de es...

Pag. 32

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y ahora que hacemos? Podemosreemplazarcadabloqueporsurespectivorankyseterminancreando dos cadenas nuevas. $S_1 \rightarrow R_1 = 12$ $S_2 \rightarrow R_2 = 30$ Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R=R_1R_2$ de longitud $2n+3$. Aplicando recursion Podemos usar recursion para obtener el suffix array de la cadena R , de es...

Pag. 33

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y ahora que hacemos? Podemosreemplazarcadabloqueporsurespectivorankyseterminancreando dos cadenas nuevas. $S_1 \rightarrow R_1 = 12$ $S_2 \rightarrow R_2 = 30$ Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R=R_1R_2$ de longitud $2n+3$. Aplicando recursion Podemos usar recursion para obtener el suffix array de la cadena R , de es...

Pag. 34

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y ahora que hacemos? Podemosreemplazarcadabloqueporsurespectivorankyseterminancreando dos cadenas nuevas. $S_1 \rightarrow R_1 = 12$ $S_2 \rightarrow R_2 = 30$ Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R=R_1R_2$ de longitud $2n+3$. Aplicando recursion Podemos usar recursion para obtener el suffix array de la cadena R , de es...

Pag. 35

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y como ordeno todo? Vamos a tener escenarios en los cuales todo se va a reducir a comparar una cantidad $O(1)$ de caracteres y una comparacion de ranks de los sufijos de S_1 y S_2 . $-x$ in S_0 vs y in S_1 : Podemos tomar el primer caracter de cada sufijo, de manera que la comparacion se reduce a comparar los pares $(S[i], \text{rank} \dots)$

Pag. 36

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y como ordeno todo? Vamos a tener escenarios en los cuales todo se va a reducir a comparar una cantidad $O(1)$ de caracteres y una comparacion de ranks de los sufijos de S_1 y S_2 . $-x$ in S_0 vs y in S_1 : Podemos tomar el primer caracter de cada sufijo, de manera que la comparacion se reduce a comparar los pares $(S[i], \text{rank} \dots)$

Pag. 37

CS3014 Estructuras de Datos Avanzadas Algoritmo ?Y como ordeno todo? Vamos a tener escenarios en los cuales todo se va a reducir a comparar una cantidad $O(1)$ de caracteres y una comparacion de ranks de los sufijos de S_1 y S_2 . $-x$ in S_0 vs y in S_1 : Podemos tomar el primer caracter de cada sufijo, de manera que la comparacion se reduce a comparar los pares $(S[i], \text{rank} \dots)$

Pag. 38

CS3014 Estructuras de Datos Avanzadas Complejidad Analisis Ya que es posible hacer la comparacion de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$ Es sencillo notar que $T(n) = O(n)$. Generalizacion Se planteo en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 39

CS3014 Estructuras de Datos Avanzadas Complejidad Analisis Ya que es posible hacer la comparacion de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$

Es sencillo notar que $T(n) = O(n)$. Generalización Se planteó en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 40

CS3014 Estructuras de Datos Avanzadas Complejidad Análisis Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$. Es sencillo notar que $T(n) = O(n)$. Generalización Se planteó en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 41

CS3014 Estructuras de Datos Avanzadas Complejidad Análisis Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$. Es sencillo notar que $T(n) = O(n)$. Generalización Se planteó en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 42

CS3014 Estructuras de Datos Avanzadas Complejidad Análisis Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$. Es sencillo notar que $T(n) = O(n)$. Generalización Se planteó en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 43

CS3014 Estructuras de Datos Avanzadas Complejidad Análisis Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$. Es sencillo notar que $T(n) = O(n)$. Generalización Se planteó en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 44

CS3014 Estructuras de Datos Avanzadas Complejidad Análisis Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma: $T(n) = T(2n/3) + O(n)$. Es sencillo notar que $T(n) = O(n)$. Generalización Se planteó en investigaciones posteriores la posibilidad de tomar modulos en vez de modulo 3. El mod...

Pag. 45

CS3014 Estructuras de Datos Avanzadas SAIS (Nong et. al.) Idea Clasificar los sufijos de S en tipo SyL . Un sufijo $Suf(S,i)$ es de tipo $-S$, si $Suf(S,i) < Suf(S,i+1)$. $-L$, si $Suf(S,i) > Suf(S,i+1)$. Un sufijo $Suf(S,i)$ es de tipo LMS si el sufijo $Suf(S,i-1)$ es de tipo Ly el sufijo $Suf(S,i)$ es de tipo S . Notemos que hay a lo mucho $n-1$ 2 sufijos de tipo LMS . Teorema (Nong et. al.) Si s...

Pag. 46

CS3014 Estructuras de Datos Avanzadas SAIS (Nong et. al.) Idea Clasificar los sufijos de S en tipo SyL . Un sufijo $Suf(S,i)$ es de tipo $-S$, si $Suf(S,i) < Suf(S,i+1)$. $-L$, si $Suf(S,i) > Suf(S,i+1)$. Un sufijo $Suf(S,i)$ es de tipo LMS si el sufijo $Suf(S,i-1)$ es de tipo Ly el sufijo $Suf(S,i)$ es de tipo S . Notemos que hay a lo mucho $n-1$ 2 sufijos de tipo LMS . Teorema (Nong et. al.) Si s...

Pag. 47

CS3014 Estructuras de Datos Avanzadas SAIS (Nong et. al.) Idea Clasificar los sufijos de S en tipo SyL . Un sufijo $Suf(S,i)$ es de tipo $-S$, si $Suf(S,i) < Suf(S,i+1)$. $-L$, si $Suf(S,i) > Suf(S,i+1)$. Un sufijo $Suf(S,i)$ es de tipo LMS si el sufijo $Suf(S,i-1)$ es de tipo Ly el sufijo $Suf(S,i)$ es de tipo S . Notemos que hay a lo mucho $n-1$ 2 sufijos de tipo LMS . Teorema (Nong et. al.) Si s...

Pag. 48

CS3014 Estructuras de Datos Avanzadas SAIS (Nong et. al.) Idea Clasificar los sufijos de S en tipo SyL . Un sufijo $Suf(S,i)$ es de tipo $-S$, si $Suf(S,i) < Suf(S,i+1)$. $-L$, si $Suf(S,i) > Suf(S,i+1)$. Un sufijo $Suf(S,i)$ es de tipo LMS si el sufijo $Suf(S,i-1)$ es de tipo Ly el sufijo $Suf(S,i)$ es de tipo S . Notemos que hay a lo mucho $n-1$ 2 sufijos de tipo LMS . Teorema (Nong et. al.) Si s...

Pag. 49

CS3014 Estructuras de Datos Avanzadas SAIS (Nong et. al.) Idea Clasificar los sufijos de S en tipo SyL . Un sufijo $Suf(S,i)$ es de tipo $-S$, si $Suf(S,i) < Suf(S,i+1)$. $-L$, si $Suf(S,i) > Suf(S,i+1)$. Un sufijo $Suf(S,i)$ es de tipo LMS

si el sufijoSuf(S,i-1)es de tipoLy el sufijoSuf(S,i)es de tipoS. Notemos que hay a lo mucho-1 2 sufijos de tipo LMS. Teorema (Nong et. al.) Si s...

Pag. 50

CS3014 Estructuras de Datos Avanzadas SAIS (Nong et. al.) Idea Clasificar los sufijos deSen tipoSyL. Un sufijoSuf(S,i)es de tipo -S, siSuf(S,i)<Suf(S,i+1). -L, siSuf(S,i)>Suf(S,i+1). Un sufijoSuf(S,i)es de tipo LMS si el sufijoSuf(S,i-1)es de tipoLy el sufijoSuf(S,i)es de tipoS. Notemos que hay a lo mucho-1 2 sufijos de tipo LMS. Teorema (Nong et. al.) Si s...

Pag. 51

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Se realiza la particion deStomando en cuenta las posiciones de los sufijos LMS y se ordenan segun(caracter,tipo)para calcular el rank. Es posible realizar esto enO(n). Reduccion Se obtiene una cadena reducidaRen base a la particion por LMS, se calcula el suffix array deRy se puede inducir el orden de...

Pag. 52

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Se realiza la particion deStomando en cuenta las posiciones de los sufijos LMS y se ordenan segun(caracter,tipo)para calcular el rank. Es posible realizar esto enO(n). Reduccion Se obtiene una cadena reducidaRen base a la particion por LMS, se calcula el suffix array deRy se puede inducir el orden de...

Pag. 53

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Se realiza la particion deStomando en cuenta las posiciones de los sufijos LMS y se ordenan segun(caracter,tipo)para calcular el rank. Es posible realizar esto enO(n). Reduccion Se obtiene una cadena reducidaRen base a la particion por LMS, se calcula el suffix array deRy se puede inducir el orden de...

Pag. 54

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Se realiza la particion deStomando en cuenta las posiciones de los sufijos LMS y se ordenan segun(caracter,tipo)para calcular el rank. Es posible realizar esto enO(n). Reduccion Se obtiene una cadena reducidaRen base a la particion por LMS, se calcula el suffix array deRy se puede inducir el orden de...

Pag. 55

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Se realiza la particion deStomando en cuenta las posiciones de los sufijos LMS y se ordenan segun(caracter,tipo)para calcular el rank. Es posible realizar esto enO(n). Reduccion Se obtiene una cadena reducidaRen base a la particion por LMS, se calcula el suffix array deRy se puede inducir el orden de...

Pag. 56

CS3014 Estructuras de Datos Avanzadas Algoritmo Particion Se realiza la particion deStomando en cuenta las posiciones de los sufijos LMS y se ordenan segun(caracter,tipo)para calcular el rank. Es posible realizar esto enO(n). Reduccion Se obtiene una cadena reducidaRen base a la particion por LMS, se calcula el suffix array deRy se puede inducir el orden de...

Pag. 57

CS3014 Estructuras de Datos Avanzadas Calculando el LCP LCP array Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array Ventajas -Ya queaesta ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos. -Ya...

Pag. 58

CS3014 Estructuras de Datos Avanzadas Calculando el LCP LCP array Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array Ventajas -Ya queaesta ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos. -Ya...

Pag. 59

CS3014 Estructuras de Datos Avanzadas Calculando el LCP LCP array Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array Ventajas -Ya

queaesta ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos. -Ya...

Pag. 60

CS3014 Estructuras de Datos Avanzadas Calculando el LCP LCP array Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array Ventajas -Ya queaesta ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos. -Ya...

Pag. 61

CS3014 Estructuras de Datos Avanzadas Calculando el LCP LCP array Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array Ventajas -Ya queaesta ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos. -Ya...

Pag. 62

CS3014 Estructuras de Datos Avanzadas Calculando el LCP ?Y como se calcula? Una implementacion ingenua podria tomar hasta $O(n^2)$ de complejidad, asi que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo. Algoritmo de Kasai Su principal observacion es que si tenemos dos sufijos que son adyacentes en a, sean x y y sus posiciones iniciales, con LCP $igu...$

Pag. 63

CS3014 Estructuras de Datos Avanzadas Calculando el LCP ?Y como se calcula? Una implementacion ingenua podria tomar hasta $O(n^2)$ de complejidad, asi que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo. Algoritmo de Kasai Su principal observacion es que si tenemos dos sufijos que son adyacentes en a, sean x y y sus posiciones iniciales, con LCP $igu...$

Pag. 64

CS3014 Estructuras de Datos Avanzadas Calculando el LCP ?Y como se calcula? Una implementacion ingenua podria tomar hasta $O(n^2)$ de complejidad, asi que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo. Algoritmo de Kasai Su principal observacion es que si tenemos dos sufijos que son adyacentes en a, sean x y y sus posiciones iniciales, con LCP $igu...$

Pag. 65

CS3014 Estructuras de Datos Avanzadas Calculando el LCP ?Y como se calcula? Una implementacion ingenua podria tomar hasta $O(n^2)$ de complejidad, asi que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo. Algoritmo de Kasai Su principal observacion es que si tenemos dos sufijos que son adyacentes en a, sean x y y sus posiciones iniciales, con LCP $igu...$

Pag. 66

CS3014 Estructuras de Datos Avanzadas Calculando el LCP ?Y como se calcula? Una implementacion ingenua podria tomar hasta $O(n^2)$ de complejidad, asi que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo. Algoritmo de Kasai Su principal observacion es que si tenemos dos sufijos que son adyacentes en a, sean x y y sus posiciones iniciales, con LCP $igu...$

Pag. 67

CS3014 Estructuras de Datos Avanzadas Calculando el LCP Pero podrian no ser adyacentes Eso es cierto, pero si su LCP es al menos $k-1$, eso implica que todas las posiciones entre ellos tambien tienen LCP de al menos $k-1$ con cualquiera de los dos. Complejidad Sikse reduce en 1 cada vez que comparamos un par de posiciones de a, este es reducido a lo mucho $n-1$ veces...

Pag. 68

CS3014 Estructuras de Datos Avanzadas Calculando el LCP Pero podrian no ser adyacentes Eso es cierto, pero si su LCP es al menos $k-1$, eso implica que todas las posiciones entre ellos tambien tienen LCP de al menos $k-1$ con cualquiera de los dos. Complejidad Sikse reduce en 1 cada vez que comparamos un par de posiciones de a, este es reducido a lo mucho $n-1$ veces...

Pag. 69

CS3014 Estructuras de Datos Avanzadas Calculando el LCP Pero podrian no ser adyacentes Eso es cierto, pero si su LCP es al menos $k-1$, eso implica que todas las posiciones entre ellos tambien tienen LCP de al menos $k-1$

con cualquiera de los dos. Complejidad Sikse reduce en 1 cada vez que comparamos un par de posiciones dea, este es reducido a lo mucho-1 veces...

Pag. 70

CS3014 Estructuras de Datos Avanzadas Calculando el LCP Pero podrian no ser adyacentes Eso es cierto, pero si su LCP es al menos k-1, eso implica que todas las posiciones entre ellos tambien tienen LCP de al menos k-1 con cualquiera de los dos. Complejidad Sikse reduce en 1 cada vez que comparamos un par de posiciones dea, este es reducido a lo mucho-1 veces...

Pag. 71

CS3014 Estructuras de Datos Avanzadas Calculando el LCP Suffix Index LCP \$ 6 - A\$ 5 0 ANA\$ 3 1 ANANA\$ 1 3 BANANA\$ 0 0 NA\$ 4 0 NANA\$ 2 2

Pag. 72

CS3014 Estructuras de Datos Avanzadas Propiedades RMQ Se puede calcular el LCP entre un par de sufijos deS mediante una consulta de RMQ sobre el LCP array. (Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas. Conclusiones Los suffix array son estructuras poderosas que permiten obten...

Pag. 73

CS3014 Estructuras de Datos Avanzadas Propiedades RMQ Se puede calcular el LCP entre un par de sufijos deS mediante una consulta de RMQ sobre el LCP array. (Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas. Conclusiones Los suffix array son estructuras poderosas que permiten obten...

Pag. 74

CS3014 Estructuras de Datos Avanzadas Propiedades RMQ Se puede calcular el LCP entre un par de sufijos deS mediante una consulta de RMQ sobre el LCP array. (Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas. Conclusiones Los suffix array son estructuras poderosas que permiten obten...

Pag. 75

CS3014 Estructuras de Datos Avanzadas Propiedades RMQ Se puede calcular el LCP entre un par de sufijos deS mediante una consulta de RMQ sobre el LCP array. (Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas. Conclusiones Los suffix array son estructuras poderosas que permiten obten...

Pag. 76

CS3014 Estructuras de Datos Avanzadas Propiedades RMQ Se puede calcular el LCP entre un par de sufijos deS mediante una consulta de RMQ sobre el LCP array. (Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas. Conclusiones Los suffix array son estructuras poderosas que permiten obten...

Pag. 77

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos suffix array -Aprendimos el algoritmo DC3 y SAIS -Aprendimos LCP array

Pag. 78

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos suffix array -Aprendimos el algoritmo DC3 y SAIS -Aprendimos LCP array

Pag. 79

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos suffix array -Aprendimos el algoritmo DC3 y SAIS -Aprendimos LCP array

Pag. 80

CS3014 Estructuras de Datos Avanzadas Resumen de la sesion -Aprendimos suffix array -Aprendimos el algoritmo DC3 y SAIS -Aprendimos LCP array

Pag. 81

CS3014 Estructuras de Datos Avanzadas Gracias

eda_slides_08_rm.pdf (27 paginas)

Pag. 1

Static Trees CS3014 - Estructura de Datos Avanzados Luciano A. Romero Calla lromeroc@utec.edu.pe 2026-1

Pag. 2

Overview 1. Goals 2. RMQ 3. RMQ & LCA 4. Solving RMQ 5. Level Ancestor 6. Related 7. Summary 2 / 20

Pag. 3

Goals 1. Goals 3 / 20

Pag. 4

Goals Goals Warming up! Problem I - Latin American Regional ACM ICPC 2017 <https://maratona.sbc.org.br/hist/2017/resultados/contest.pdf> Do it yourself! (homework) <https://judge.beecrowd.com/en/problems/view/2703> 4 / 20

Pag. 5

Goals Goals Warming up! Problem I - Latin American Regional ACM ICPC 2017 <https://maratona.sbc.org.br/hist/2017/resultados/contest.pdf> Do it yourself! (homework) <https://judge.beecrowd.com/en/problems/view/2703> 4 / 20

Pag. 6

Goals Goals - Solve the RMQ problem query in $O(1)$ - Explore different structures and analyze their performance. - Be able to reduce the LCA problem to RMQ and solve it. - Solve the Level Ancestor (LA) problem. 5 / 20

Pag. 7

Goals Goals - Solve the RMQ problem query in $O(1)$ - Explore different structures and analyze their performance. - Be able to reduce the LCA problem to RMQ and solve it. - Solve the Level Ancestor (LA) problem. 5 / 20

Pag. 8

Goals Goals - Solve the RMQ problem query in $O(1)$ - Explore different structures and analyze their performance. - Be able to reduce the LCA problem to RMQ and solve it. - Solve the Level Ancestor (LA) problem. 5 / 20

Pag. 9

Goals Goals - Solve the RMQ problem query in $O(1)$ - Explore different structures and analyze their performance. - Be able to reduce the LCA problem to RMQ and solve it. - Solve the Level Ancestor (LA) problem. 5 / 20

Pag. 10

Goals Goals Problem Scenario Preprocessing Space Query Time Range Minimum Query (RMQ) $O(n)O(n)O(1)$ Lowest Common Ancestor (LCA) $O(n)O(n)O(1)$ Level Ancestor (LA) $O(n)O(n)O(1)$ Goal: Achieve optimal linear preprocessing and constant time queries across all structures. 6 / 20

Pag. 11

RMQ 2. RMQ 7 / 20

Pag. 12

RMQ RMQ Problem Given an array A of n numbers (to preprocess). In a query, the goal is to find the minimum element in a range spanned by $A[i]$ and $A[j]$: $RMQ(i,j) = \arg \min\{A[i], A[i+1], \dots, A[j]\} = k$ where $i \leq k \leq j$ and $A[k]$ is minimized 8 / 20

Pag. 13

RMQ & LCA 3. RMQ & LCA 3.1 Cartesian trees 3.2 LCA to 1RMQ: Euler Tour 9 / 20

Pag. 14

RMQ & LCA Cartesian trees Cartesian trees Rules: - Root = Minimum element $A[m]$ - Left / Right subtrees built recursively. - Preserves min-heap property. Identity $RMQA(i,j) = LCA(T(i,j))$ $A = [7, 8, 2, 6, 9, 4]$ 2 7 8 4 6 9 10 / 20

Pag. 15

RMQ & LCA Cartesian trees Cartesian trees Linear time algorithm - Scan array left to right. - Walk up the right spine until finding $A[v] < A[i]$. - Insert $A[i]$ as right child; old subtree becomes left child of $A[i]$. - Amortized Cost: Nodes enter/leave spine ≤ 1 time $\Rightarrow O(n)$. 2 4 7 5 11 / 20

Pag. 16

RMQ & LCA LCA to 1RMQ: Euler Tour LCA to 1RMQ: Euler Tour A B D C Euler Tour (E): A B D B A C A Depth Array (D): 0 1 2 1 0 1 0 The 1-Property Consecutive depth steps change by exactly +1 or -1: $|D[i] - D[i-1]| = 1$ $LCA(u, v) = \min \text{value in } D \text{ between indices of } u \text{ and } v$. 12 / 20

Pag. 17

Solving RMQ 4. Solving RMQ 13 / 20

Pag. 18

Solving RMQ Solving RMQ Block partition size: $b = \lceil \sqrt{2 \log_2 n} \rceil$. 1. Macro-Structure (Across Blocks) - Sparse Table over block minimums. - Space: $O(n \cdot b \log n) = O(n)$. 2. Micro-Structure (Inside Blocks) - Max unique shapes $= 2b - 1 = O(\sqrt{n})$. - Precompute block lookup tables. - Space: $O(n \cdot b) = O(n)$. Total Performance: $O(n)$ Space / $O(1)$ Query Time 14 / 20

Pag. 19

Level Ancestor 5. Level Ancestor 15 / 20

Pag. 20

Level Ancestor Level Ancestor Strategy / Name Preprocessing Query Time Core Technique Full Table Lookup $O(n^2)$ $O(1)$ Complete DP lookup table Jump Pointers $O(n \log n)$ $O(\log n)$ Binary lifting (powers of 2) Longest Path Decomposition $O(n)$ $O(\sqrt{n})$ Disjoint root-to-leaf path arrays Ladder Decomposition $O(n)$ $O(\log n)$ Overlapping path extensions (doubled length) Jump + Ladder $O(n \log n)$

Pag. 21

Related 6. Related 17 / 20

Pag. 22

Related Related Dynamic Segment tree, Fenwick tree, ... 18 / 20

Pag. 23

Summary 7. Summary 19 / 20

Pag. 24

Summary Summary Problem Scenario Preprocessing Space Query Time Range Minimum Query (RMQ) $O(n)$ $O(n)$ $O(1)$ Lowest Common Ancestor (LCA) $O(n)$ $O(n)$ $O(1)$ Level Ancestor (LA) $O(n)$ $O(n)$ $O(\log n)$, $O(1)$ Goal: Achieve optimal linear preprocessing and constant time queries across all structures. 20 / 20

Pag. 25

Thanks

Pag. 26

References References 22 / 20

Pag. 27

Acknowledgements Acknowledgements The course slides are based on the lectures from previous editions and on similar lectures elsewhere. List of credits: Erick Demaine, Keith Schwarz 23 / 20

Cierre

Si tienes poco tiempo, aprende en este orden:

1. Sparse table y RMQ.
2. LCA con binary lifting y Euler tour.

3. Suffix array y LCP.
4. Pattern matching como intervalo en suffix array.
5. Subcadenas distintas y longest common substring.
6. LCP arbitrario con RMQ.
7. Ideas de DC3/SA-IS solo para explicar construccion lineal.